

UNIVERSITÀ DEGLI STUDI DI MILANO

FACOLTÀ DI SCIENZE E TECNOLOGIE

CORSO DI LAUREA TRIENNALE IN INFORMATICA



INTEGRAZIONE DI METODI DI MACHINE

LEARNING IN SQL

Relatore: Prof. Alfio Ferrara

Correlatore: Dott.sa Darya Shlyk

Tesi di Laurea di:

Riccardo Fidanza

Matr. 941677

anno accademico 2022/2023

Indice

Introduzione	3
1.1 Contesto	3
1.2 Obiettivo della tesi	4
1.3 Struttura della tesi	5
Machine Learning nei Database	7
2.1 Vantaggi dei database relazionali	7
2.2 Flessibilità di SQL per Algoritmi nel DB	8
2.2.1 Breve introduzione di SQL	8
2.2.2 Vantaggi ricavati dall'uso di SQL	9
2.3 Soluzioni proposte in letteratura	10
2.3.1 MASQ	10
2.3.2 sql4ml	10
2.3.3 Exasol	11
2.3.4 Aida - (Abstraction for Advanced In-Database Analytics)	11
2.3.5 SQLFlow	12
2.3.6 confronto con il progetto di tesi	12
Processo di produzione del pacchetto	13
3.1 Processo incrementale	13
3.2 Fasi	13
Analisi dei requisiti del progetto	15
4.1 Requisiti funzionali del pacchetto	15
4.1.1 Integrazione con il database	15
4.1.2 Adattabilità	16
4.1.3 Memorizzazione del modello addestrato nel database	16
4.1.4 Logica predittiva direttamente nel database	16
4.1.5 Accuratezza comparabile a Scikit-learn	17
4.1.6 Supportare due rappresentazioni dei dati	17
4.2 Requisiti non funzionali del pacchetto	18
4.3 Processo di utilizzo del pacchetto	19
Progettazione e sviluppo	21
5.1 Architettura del pacchetto	21
5.1.1 Struttura generale del pacchetto	21
5.2 Progettazione dei moduli	24
5.2.1 Dettagli sulla progettazione dei moduli del pacchetto	24
5.2.2 Dettagli sulla progettazione delle interfacce dei moduli	25

5.3 Sviluppo	27
5.3.1 Classe Builder	27
5.3.2 Definizione e costruttori delle Classi LogisticRegressionRE e LogisticRegressionCOO	28
5.3.2 Addestramento	28
5.3.3 Predizione	33
Valutazione	36
6.1 Dataset utilizzati per la valutazione	36
6.1.1 Iris dataset	37
6.1.2 Zoo dataset	38
6.2 Valutazione dell'accuratezza predittiva	40
6.2.1 Metodo di valutazione	40
6.2.2 Confronto con scikit-learn	40
6.2.3 Risultati	40
6.3 Valutazione delle prestazioni	41
6.3.1 Metodo di valutazione	41
6.3.2 Grafici dei tempi	41
Conclusione e sviluppi futuri	47
BIBLIOGRAFIA	49

Capitolo 1

Introduzione

1.1 Contesto

Il campo del Machine Learning ha conosciuto una crescente applicazione in molteplici settori. Tuttavia, l'implementazione pratica di modelli di machine learning su grandi dataset può essere problematica a causa di diversi fattori, tra cui la complessità computazionale, la gestione della memoria e la scalabilità. In particolare, la necessità di estrarre e preparare i dati per la predizione può richiedere notevoli risorse e tempo, oltre a comportare un rischio aggiuntivo per la sicurezza dei dati.

L'integrazione dei modelli di machine learning in SQL, il linguaggio standard per l'interazione con i database relazionali, emerge come un campo di studio promettente, consentendo l'esecuzione diretta di calcoli sui dati memorizzati nei database. Di conseguenza, c'è un crescente interesse nello sviluppo di soluzioni che sfruttano la potenza e la scalabilità dei database. Questo approccio offre numerosi vantaggi, tra cui la riduzione della necessità di trasferire grandi quantità di dati tra diversi sistemi, la possibilità di eseguire analisi in tempo reale sui dati in continua evoluzione e la facilità di gestione e manutenzione dei modelli all'interno dell'infrastruttura esistente.

Nonostante l'intrinseca semplicità di SQL, scrivere query SQL rilevanti ed efficienti è noto per essere difficile [1], l'integrazione di algoritmi di machine learning complessi nelle query SQL può essere un compito non banale, richiedendo una profonda comprensione delle strutture dei dati e delle operazioni computazionali.

Nella mia tesi, mi concentrerò sull'approfondimento di questo tema, proponendo un metodo per l'integrazione di modelli di machine learning nell'ambiente SQL e valutando le loro prestazioni

su dataset di dimensioni reali e complesse. L'obiettivo è fornire una nuova prospettiva sulle potenzialità derivanti dall'incrocio tra machine learning e SQL, aprendo così la strada a nuove opportunità per l'analisi avanzata dei dati e l'estrazione di informazioni significative.

1.2 Obiettivo della tesi

Il presente lavoro si propone di sviluppare un pacchetto Python il cui scopo principale è integrare metodi di machine learning direttamente nell'ambiente SQL. Questo obiettivo nasce dalla necessità di condurre analisi predittive sui dati archiviati nei database, senza dover ricorrere all'estrazione preliminare dei dati stessi.

Il pacchetto ha lo scopo di fornire agli utenti uno strumento flessibile e intuitivo che consenta loro di sfruttare appieno le potenzialità dei database per l'analisi dei dati, garantendo al contempo un'elevata accuratezza predittiva e prestazioni ottimali.

Tra i requisiti fondamentali del pacchetto vi è la possibilità per gli utenti di eseguire analisi predittive direttamente nel database, eliminando così l'overhead e i costi associati alla gestione e pre-elaborazione dei dati derivanti dall'estrazione e trasferimento dei dati dal database all'ambiente di sviluppo.

Inoltre, il pacchetto deve sfruttare appieno le capacità dei moderni sistemi DBMS nella gestione ed elaborazione di grandi volumi di dati, al fine di migliorare l'efficienza e la scalabilità dei calcoli.

Un altro aspetto fondamentale è la portabilità che si intende perseguire utilizzando il linguaggio standard adottato da maggior parte dei sistemi dbms relazionali per la definizione della logica inferenziale degli algoritmi di machine learning.

Il pacchetto cerca di assicurare l'accuratezza predittiva gestendo la fase di training con la libreria ml specializzata. Mentre la logica inferenziale è interamente integrata nel sistema dbms.

Il pacchetto supporta due rappresentazioni dei dati, il formato Coordinat Format List (COO) per dati sparsi e il formato relazionale per dati densi. Questa flessibilità consentirà al pacchetto di adattarsi a una vasta gamma di scenari e tipologie di dataset, offrendo agli utenti la possibilità di utilizzarlo in molteplici contesti applicativi.

Infine, viene data particolare importanza alla facilità d'uso e all'estendibilità del pacchetto. Si mira a garantire che il pacchetto sia intuitivo e accessibile anche per gli utenti meno esperti, consentendo loro di utilizzare le funzionalità di machine learning senza la necessità di una profonda conoscenza tecnica. Inoltre, il pacchetto sarà progettato per essere facilmente estendibile, consentendo l'aggiunta di nuovi algoritmi di machine learning e nuove funzionalità in futuro.

1.3 Struttura della tesi

L'elaborato si dirama nei seguenti capitoli:

- **Introduzione:** introduce il tema principale della tesi, fornendo una panoramica generale del contesto in cui si inserisce il lavoro di ricerca. Vengono delineati gli obiettivi e le motivazioni alla base dello studio. Si fornisce un'anteprima dei capitoli successivi e si delinea il percorso che verrà seguito nel documento.
- **Machine Learning nei Database:** offre una panoramica della letteratura esistente relativa all'integrazione di tecniche di machine learning nei database. Si esplorano le principali tendenze, gli approcci e le sfide affrontate dalla comunità scientifica e industriale in questo campo. Viene analizzato il ruolo cruciale che svolgono i database nel contesto dell'analisi dei dati e del machine learning.
- **Processo di produzione del pacchetto:** offre una spiegazione sul processo di realizzazione del pacchetto, descrivendo l'approccio e le fasi del processo.

- **Analisi dei requisiti del progetto:** il capitolo analizza in dettaglio i requisiti del progetto, inclusi i requisiti funzionali e non funzionali. Si delineano le esigenze, le caratteristiche desiderate del pacchetto e le limitazioni da considerare durante lo sviluppo. Si fornisce una panoramica completa dei requisiti che guidano il processo di progettazione e sviluppo del pacchetto.
- **Progettazione e Sviluppo:** approfondisce la progettazione e lo sviluppo del pacchetto Python per l'integrazione dei modelli di machine learning in SQL. Si descrivono le scelte architetturali, le componenti del sistema, le strutture dati e le funzionalità implementate. Si illustrano i dettagli tecnici del pacchetto e si discutono le sfide affrontate durante il processo di sviluppo, attraverso l'ausilio di esempi.
- **Valutazioni:** vengono presentati i risultati delle valutazioni effettuate sul pacchetto implementato. Si analizzano l'accuratezza e le prestazioni del pacchetto attraverso test sperimentali e confronti con implementazioni equivalenti presenti in altre librerie di machine learning. Si discutono i risultati ottenuti.
- **Conclusione e sviluppi futuri:** riassume le principali conclusioni e presenta possibili sviluppi futuri.

Capitolo 2

Machine Learning nei Database

Esistono numerosi metodi esistenti per il machine learning, disponibili in linguaggi come R, Python o Java e in ambienti come CRAN, PyPi e Weka. Tuttavia, la maggior parte di questi metodi e strumenti funziona solo per set di dati che possono essere elaborati in memoria. I problemi dei big data nel settore ML iniziano con volumi di pochi gigabyte [2].

Le moderne piattaforme di machine learning sono principalmente progettate per supportare il parallelismo dei dati, in cui un insieme di calcoli quasi identici viene eseguito simultaneamente su un insieme di unità computazionali [3, 4]. Tuttavia, per affrontare modelli di grandi dimensioni emergono limitazioni dovute alla capacità delle risorse computazionali di gestire dati che superano la loro capacità di memoria [3]. In alternativa si può considerare di implementare il parallelismo di modello, in particolare sfruttando i Database relazionali. Le diverse parti del modello possono essere memorizzate in un insieme di tabelle, e i calcoli sul modello parziale possono spesso essere espressi attraverso poche query SQL [3].

Di fatti, l'integrazione delle funzionalità di machine learning nei sistemi di database relazionali è un'area che ha ricevuto molta attenzione [5, 6, 7, 8] e considerando che un numero sempre crescente di organizzazioni sta installando grandi data warehouse utilizzando la tecnologia dei database relazionali [9], avere un metodo per eseguire il machine learning direttamente all'interno dei database potrebbe risultare conveniente.

2.1 Vantaggi dei database relazionali

Considerando che il machine learning tradizionale opera su dati tabulari e che i sistemi di gestione dei database sono parte integrante di ogni applicazione e attività incentrata sui dati [10], utilizzare i DBMS come ambiente per il machine learning porta i seguenti benefici:

- **Efficienza:** Un punto di forza della tecnologia relazionale è la sua efficienza nell'eseguire operazioni anche su grandi quantità di dati. Grazie a ottimizzazioni e algoritmi sofisticati, i RDBMS possono garantire tempi di risposta rapidi e prestazioni affidabili in situazioni di carico elevato. Difatti, i sistemi relazionali dispongono già di componenti software come un ottimizzatore di query basato sui costi per aiutare nell'esecuzione di calcoli efficienti [11].
- **Robustezza e rapidità:** I calcoli nei sistemi di gestione di database relazionali (RDBMS) sono stati studiati per oltre trent'anni e sono rapidi e robusti [3], di fatti l'ottimizzatore delle query, è molto efficace nell'ottimizzazione dei calcoli distribuiti [2, 12].
- **Interfaccia di programmazione dichiarativa:** I RDBMS forniscono un'interfaccia di programmazione dichiarativa [3]. Questo permette all'utente di specificare solo quale deve essere il risultato e lasciando al DBMS l'ottimizzazione e le decisioni effettive su come eseguire l'interrogazione [13].
- **Sicurezza dei dati:** ogni copia di un database è essenzialmente un rischio di fughe di informazioni, duplicati, conflitti di versione e incoerenze. Pertanto, se i dati possono rimanere nel database per l'elaborazione predittiva, si ottiene una maggiore integrità e riservatezza dei dati [14].

2.2 Flessibilità di SQL per Algoritmi nel DB

2.2.1 Breve introduzione di SQL

SQL è il linguaggio standard per il recupero e la manipolazione dei dati relazionali [15]. È stato sviluppato da IBM nei primi anni '70 e successivamente standardizzato dall' ANSI (American National Standards Institute) e dall' ISO (International Organization for Standardization). Questo

significa che la sintassi e le funzionalità di base di SQL sono uniformi e interoperabili tra diverse implementazioni e sistemi di gestione di database. Ciò consente di scrivere script SQL che possono essere eseguiti su diversi database, rendendo SQL un linguaggio di query portatile e versatile utilizzato in una vasta gamma di contesti e applicazioni.

In particolare, SQL è Turing complete [15,16], il che significa che ha la potenza computazionale necessaria per eseguire una vasta gamma di operazioni e calcoli complessi.

2.2.2 Vantaggi ricavati dall'uso di SQL

Sebbene SQL, essendo un linguaggio dichiarativo, possa rendere non banale la scrittura di una query per eseguire algoritmi complessi, vi sono comunque benefici:

- **Calcolo vicino ai dati:** Nel machine learning, il continuo ritraining di un modello garantisce previsioni accurate basate sui dati più recenti come input di addestramento. Tuttavia, per recuperare i dati più recenti da un database, è necessaria un'estrazione che richiede tempo [17]. Con gli algoritmi SQL i dati rimangono nel database [15], permettendo quindi di evitarne l'estrazione.
- **Flessibilità:** gli algoritmi SQL possono essere adattati specificamente ai diversi casi d'uso e modificati in base alle esigenze specifiche. Inoltre, gli algoritmi SQL richiedono solo permessi minimi agli utenti. Questo significa che anche gli utenti con privilegi di sola lettura possono eseguire una vasta gamma di calcoli senza necessità di autorizzazioni aggiuntive.
- **Alto livello di astrazione:** SQL è un linguaggio che astrae fortemente dai dettagli architettonici di un DBMS [15].
- **Specifica dichiarativa:** SQL consente la specifica dichiarativa delle proprietà che le tuple di risultato devono soddisfare [18].

- **Portabilità:** SQL è uno standard internazionale supportato da centinaia di tipi di database in milioni di sistemi informativi [14]. Se gli algoritmi utilizzano un sottoinsieme SQL comune supportato da più fornitori di DBMS, possono essere eseguiti su altri DBMS senza modifiche [15].

2.3 Soluzioni proposte in letteratura

In letteratura sono presenti diverse soluzioni che sfruttano i DBMS per il machine learning.

2.3.1 MASQ

Una possibile soluzione proposta in letteratura per affrontare questo problema è il framework MASQ (Machine Learning AS Query), un progetto open source con licenza MIT, che consente di eseguire l'inferenza dei modelli di ML direttamente sui DBMS attraverso l'utilizzo di SQL standard [10]. In particolare MASQ compila modelli addestrati e pipeline ML implementati in scikit-learn direttamente in SQL standard [10], consentendo l'esecuzione delle query su qualsiasi DBMS senza l'uso di sintassi specifiche del fornitore.

MASQ è progettato per ridurre il debito tecnico nelle applicazioni di machine learning, fornendo un modo efficiente per eseguire l'inferenza dei modelli direttamente sui dati nel database, riducendo così i costi di movimento dei dati e sfruttando le funzionalità avanzate dei DBMS.

2.3.2 sql4ml

sql4ml è un framework per esprimere algoritmi di machine learning in un sistema di gestione di database relazionali (RDBMS) [5].

In particolare l'utente definisce la funzione obiettivo di un modello di machine learning utilizzando una query SQL che sql4ml traduce automaticamente in un grafo TensorFlow, che può

essere automaticamente differenziato e ottimizzato per apprendere i pesi del modello [5] semplificando così il processo di addestramento.

L'approccio proposto è più espressivo rispetto all'utilizzo di algoritmi ML già pronti, ma astrae i dettagli del processo di addestramento [5].

2.3.3 Exasol

Una soluzione all'avanguardia propone un framework estensibile per il machine learning all'interno di Exasol, un database analitico ad elaborazione parallela ad alte prestazioni. Il framework estende le capacità del motore SQL di Exasol incorporando la libreria di machine learning Python Scikit-Learn [19].

In particolare viene proposta la funzione `CREATE MODEL` che permette di addestrare modelli di machine learning direttamente nel database [19] e la funzione `PREDICT` che permette di eseguire la previsione dei valori di una variabile target utilizzando un modello precedentemente addestrato.

Questa integrazione elimina la necessità per lo spostamento dei dati tra sistemi diversi, riducendo la latenza e migliorando l'efficienza complessiva del flusso di lavoro di analisi [19].

2.3.4 Aida - (Abstraction for Advanced In-Database Analytics)

AIDA emula la sintassi e la semantica dei pacchetti di data science popolari ma esegue in modo trasparente le trasformazioni e i calcoli richiesti all'interno del RDBMS [20].

Questo approccio non è limitato agli algoritmi di machine learning, ma riflette perfettamente l'idea di utilizzare i database relazionali per eseguire le operazioni necessarie senza esportare i dati all'esterno.

2.3.5 SQLFlow

SQLFlow è un framework progettato per sviluppare efficientemente flussi di lavoro di machine learning end-to-end utilizzando SQL. Consente agli sviluppatori di scrivere programmi SQL che possono integrare diverse fasi di machine learning, tra cui addestramento di modelli, previsione, spiegazione dei modelli e elaborazione dei dati.

SQLFlow adotta un'altra strategia per funzionare come un ponte su vari sistemi di database, inclusi MySQL, Apache Hive e Alibaba MaxCompute, e motori di machine learning come TensorFlow, XGBoost e scikit-learn [21] .

2.3.6 confronto con il progetto di tesi

Le soluzioni presentate ed il progetto di tesi, condividono un obiettivo comune di integrare operazioni di machine learning all'interno di database relazionali, consentendo agli utenti di eseguire analisi predittive direttamente sui dati archiviati senza doverli trasferire in un ambiente esterno. Tutte queste soluzioni mirano a ridurre l'overhead e i costi associati all'estrazione e alla gestione dei dati, fornendo un ambiente più efficiente e scalabile per l'analisi dei dati.

Capitolo 3

Processo di produzione del pacchetto

3.1 Processo incrementale

Il processo di produzione del pacchetto ha adottato un approccio incrementale, consentendo di ottenere benefici sostanziali. Con questo metodo, il pacchetto viene sviluppato gradualmente, rilasciando parti e componenti step by step per costruire progressivamente il pacchetto completo. Inoltre, questo approccio si adatta con agilità alla volatilità dei requisiti, consentendo flessibilità durante il processo di sviluppo. Ogni iterazione del processo integrava nuove tecniche (o migliorava quelle già presenti).

3.2 Fasi

Il processo di produzione del pacchetto è suddiviso in più fasi, ognuna delle quali contribuisce al completamento graduale del pacchetto.

- **Analisi dei requisiti:** vengono identificati e analizzati i requisiti iniziali del pacchetto, sia funzionali che non funzionali. Questo coinvolge una comprensione approfondita delle esigenze degli utenti e del contesto operativo del pacchetto. È importante notare che i requisiti possono evolversi nel tempo in risposta a cambiamenti nel contesto operativo o nei feedback degli utenti.
- **Progettazione:** la fase di progettazione si concentra sulla strutturazione dell'incremento del pacchetto sotto diversi livelli di dettaglio. Questo include la definizione dell'architettura del software, la progettazione delle interfacce utente e l'organizzazione

dei moduli e dei componenti del pacchetto. La progettazione mira a garantire che il pacchetto sia ben strutturato, manutenibile ed estensibile.

- **Sviluppo:** durante la fase di sviluppo, avviene l'implementazione pratica dell'incremento del pacchetto. Questo coinvolge la scrittura del codice sorgente, la creazione di test unitari e l'integrazione dei diversi moduli e componenti del pacchetto. Lo sviluppo si basa sulla progettazione definita nelle fasi precedenti e mira a tradurre i requisiti identificati in funzionalità concrete del pacchetto.
- **Valutazione:** nella fase di valutazione, si esamina l'efficacia e l'efficienza dell'approccio di integrazione del machine learning direttamente nel database.

Capitolo 4

Analisi dei requisiti del progetto

L'analisi dei requisiti del progetto costituisce una fase cruciale nello sviluppo del pacchetto, poiché definisce le specifiche funzionali e non funzionali necessarie per guidare il suo sviluppo. Questa sezione del documento espone i requisiti funzionali che hanno orientato il design e l'implementazione del pacchetto, garantendo un'integrazione efficace e una user experience ottimale. Inoltre, verrà illustrato il funzionamento del pacchetto e come viene utilizzato.

4.1 Requisiti funzionali del pacchetto

Al fine di realizzare questa integrazione in modo efficace e efficiente, sono stati definiti e rispettati i seguenti requisiti funzionali, che hanno guidato lo sviluppo del pacchetto.

4.1.1 Integrazione con il database

Il pacchetto deve fornire un'interfaccia intuitiva e flessibile per connettersi ed interagire con il database nel modo più semplice possibile, senza che l'utente necessiti di occuparsi dei dettagli interni, offrendo un'esperienza utente fluida e intuitiva, indipendentemente dalla complessità del sistema sottostante. Inoltre, dovrebbe essere in grado di adattarsi dinamicamente alle varie esigenze dell'utente, garantendo una gestione efficace delle operazioni di accesso e manipolazione dei dati, senza compromettere la sicurezza e l'affidabilità del sistema.

4.1.2 Adattabilità

Il pacchetto deve garantire l'adattabilità con i principali Database Management System, consentendo all'utente di connettersi e interagire con diversi sistemi di database in modo fluido e semplice. L'obiettivo è di garantire l'utilizzo senza alcuna restrizione in relazione al tipo di database utilizzato.

Questo contribuisce a migliorare l'accessibilità nell'uso del pacchetto e permette agli utenti di operare senza doversi preoccupare delle differenze tra i diversi sistemi di database, migliorando così l'esperienza complessiva. La capacità di interoperabilità con una vasta gamma di database riduce la necessità di cambiare le procedure o il codice quando si passa da un sistema di database all'altro, semplificando così lo sviluppo e la manutenzione delle applicazioni.

4.1.3 Memorizzazione del modello addestrato nel database

I parametri del modello addestrato devono essere salvati nel database in un'opportuna tabella, così da garantire una gestione centralizzata ed efficiente delle informazioni necessarie per le operazioni di predizione, consentendo un accesso rapido e ottimizzato durante l'elaborazione dei dati.

4.1.4 Logica predittiva direttamente nel database

Il pacchetto deve fornire la possibilità di eseguire la logica predittiva direttamente all'interno del database, così da ottenere i vantaggi precedentemente citati nel secondo capitolo. Di conseguenza, gli algoritmi devono essere implementati attraverso query SQL che operano su una tabella contenente i dati su cui effettuare la predizione e su una tabella che contiene i parametri del modello addestrato.

4.1.5 Accuratezza comparabile a Scikit-learn

Il pacchetto deve garantire che l'accuratezza dei modelli implementati nel pacchetto sia paragonabile a quella ottenuta con strumenti di riferimento, come Scikit-learn. Questo assicura che il pacchetto fornisca risultati di predizione affidabili e competitivi.

4.1.6 Supportare due rappresentazioni dei dati

Il pacchetto deve supportare due possibili rappresentazioni dei dati, così da ottimizzare le prestazioni in caso i dati sui quali effettuare la predizione siano densi o sparsi.

Dati densi: si riferiscono a un tipo di dati in cui la maggior parte degli elementi o delle celle ha un valore associato. In altre parole, in un set di dati densi, la maggior parte delle voci è presente e contiene informazioni significative. In questo caso la rappresentazione dei dati è quella tradizionale dei database relazionali, ove ogni riga rappresenta un'istanza specifica ed ogni colonna rappresenta un attributo o caratteristica dei dati.

Segue un esempio di questa rappresentazione dei dati.

Nome	Età	Occupazione	Hobby
Alice	25	Ingegnere	Fotografia
Bob	30	Designer	Escursionismo

Dati sparsi: i dati sono memorizzati come in una matrice sparsa, ovvero, una matrice che ha pochi elementi diversi da zero [22]. In questo caso la rappresentazione dei dati usata è formata da un insieme di triple: riga, colonna, valore. Tale rappresentazione è nota come Coordinate Format

List (COO), e consente di memorizzare esclusivamente i dati non nulli, riducendo lo spazio occupato rispetto alla tradizionale rappresentazione dei dati.

Segue un esempio di questa rappresentazione dei dati.

Riga	Colonna	Valore
1	1	1
1	2	1
2	2	1

4.2 Requisiti non funzionali del pacchetto

Per garantire il successo e l'efficacia del pacchetto, è essenziale considerare una serie di requisiti non funzionali:

- **Prestazioni:** Il pacchetto deve essere ottimizzato per garantire tempi di risposta rapidi. Le operazioni di addestramento del modello e di predizione dei dati devono essere eseguite in modo efficiente, anche su grandi volumi di dati, per assicurare una rapida elaborazione delle richieste degli utenti.
- **Affidabilità:** Il pacchetto deve funzionare in modo coerente e accurato nel tempo, fornendo risultati attendibili agli utenti, senza rischi eccessivi di errori o malfunzionamenti.
- **Usabilità:** È essenziale garantire che gli utenti possano utilizzare facilmente tutte le funzionalità del pacchetto senza difficoltà. Una buona usabilità è fondamentale per massimizzare l'efficacia e l'efficienza dell'utilizzo.

- **Scalabilità:** Il pacchetto dovrebbe essere in grado di gestire volumi crescenti di dati senza compromettere la sua affidabilità.

4.3 Processo di utilizzo del pacchetto

In questa sezione viene delineato il flusso di lavoro del pacchetto, evidenziando le fasi essenziali del processo. Tale panoramica fornisce una comprensione approfondita di come viene utilizzato il pacchetto.

Connessione al database: la prima fase coinvolge la connessione al database, non coinvolge l'utilizzo del pacchetto, consente l'accesso all'ambiente relazionale su cui il pacchetto opererà.

Preparazione dei dati: Questa fase non include l'utilizzo del pacchetto e viene fatta in autonomia dall'utente. Prevede la preparazione dei dati utilizzati per addestrare il modello (all'esterno del database) ed i dati sui quali effettuare la predizione i quali devono essere inseriti in una tabella nel database, se non già presenti.

Scelta del modello: dopo la preparazione dei dati, viene effettuata la fase di scelta del modello più adatto per il problema specifico. La selezione del modello è guidata dagli obiettivi dell'analisi, dalla natura dei dati (sparsi o densi) e dalle esigenze specifiche dell'utente. Questa fase è cruciale poiché il modello scelto influenzerà direttamente le prestazioni e l'efficacia dell'analisi dei dati e delle previsioni.

Addestramento del modello: una volta che i dati sono pronti, il modello viene addestrato utilizzando il metodo fornito dal pacchetto ed i dati specifici per la fase di training. Al termine di questa fase, si ottiene un modello addestrato che verrà inserito in un'opportuna tabella nel database, per poi essere utilizzato nella fase di predizione dei dati.

Predizione dei dati: una volta che il modello è stato addestrato, può essere utilizzato per effettuare predizioni su nuovi dati. Il pacchetto esegue la predizione utilizzando il modello addestrato e restituisce i risultati all'utente.

Capitolo 5

Progettazione e sviluppo

La progettazione e lo sviluppo di SQML sono stati guidati da un obiettivo primario: fornire agli utenti uno strumento robusto e flessibile per l'integrazione di algoritmi di machine learning nei database relazionali. Questa sezione esplorerà il processo di progettazione e sviluppo di SQML.

5.1 Architettura del pacchetto

In questa sezione viene descritta l'architettura del pacchetto, evidenziando la sua struttura generale e le dipendenze del pacchetto.

5.1.1 Struttura generale del pacchetto

La struttura del pacchetto (denominato SQML) è concepita per fornire un'interfaccia che consenta agli utenti di utilizzare algoritmi di machine learning integrati in SQL.

Questa progettazione è mirata a favorire una facile estensione e manutenzione del pacchetto.

Il pacchetto è strutturato come segue:

- Directory principale sqml: una cartella di livello superiore che contiene tutti gli elementi del pacchetto.
- sotto-directory build: contiene file generati durante la costruzione.
- sotto-directory sqml: contiene il codice sorgente.

- script `setup.py`: utilizzato per definire le informazioni di configurazione di un pacchetto Python.

Struttura del pacchetto:

```

sqlml (directory principale)
|
├── build (directory)
|   └── (file generati durante la costruzione)
|
├── sqlml (directory)
|   ├── knn.py
|   │   ├── KnnClassifierRE.py
|   │   └── KnnClassifierCOO.py
|   │
|   ├── decisionTree.py
|   │   ├── DecisionTreeRE.py
|   │   └── DecisionTreeCOO.py
|   │
|   ├── randomForest.py
|   │   ├── RandomForestRE.py
|   │   └── RandomForestCOO.py
|   │
|   ├── logistic_regression.py
|   │   ├── LogisticRegressionRE.py
|   │   └── LogisticRegressionCOO.py
|   │
|   ├── __init__.py
|   │
|   └── (altre risorse del pacchetto)
|
└── setup.py

```

All'interno del pacchetto, i vari algoritmi di machine learning sono suddivisi in moduli, ognuno dei quali rappresenta una tecnica specifica. Ogni modulo contiene due classi distinte, una per ciascuna delle due rappresentazioni dei dati supportate. Ad esempio, nel modulo dedicato al classificatore K-Nearest Neighbors (KNN), troviamo la classe `KnnClassifierRE`, che implementa il KNN utilizzando la rappresentazione relazionale dei dati, e la classe `KnnClassifierCOO`, che lo implementa utilizzando la rappresentazione COO.

5.1.2 Tecnologie e framework utilizzati

Il pacchetto si basa sull'utilizzo dei DBMS (Sistemi di Gestione di Database) per la gestione dei database relazionali e per sfruttare la potenza del linguaggio SQL durante la fase predittiva.

Il pacchetto SQML si basa su un insieme di framework fondamentali, senza i quali non può funzionare correttamente. Per utilizzare SQML è necessario avere installati i seguenti pacchetti:

- NumPy: libreria fondamentale per il calcolo scientifico in Python [23].
- Pandas: pandas è una libreria che fornisce strutture dati ad alte prestazioni e facili da usare e strumenti di analisi dei dati per il linguaggio di programmazione Python [24].
- scikit-learn: uno degli strumenti più utilizzati per implementare modelli di machine learning in Python. SQML dipende fortemente da scikit-learn, in particolare, viene utilizzato per l'addestramento.
- SQLAlchemy: SQLAlchemy è un toolkit SQL in Python e un Object Relational Mapper (ORM) che offre agli sviluppatori di applicazioni tutta la potenza e la flessibilità di SQL [25]. Offre un' API per eseguire query SQL e gestire connessioni al database in modo flessibile. SQML utilizza SQLAlchemy per facilitare l'integrazione tra i modelli di machine learning e i database relazionali. Questo approccio facilita lo sviluppo di

applicazioni Python che devono interagire con database relazionali, rendendo il codice più leggibile e manutenibile. Inoltre, un'altra caratteristica fondamentale è che SQLAlchemy supporta una varietà di motori di database, consentendo agli sviluppatori di connettersi a una vasta gamma di database relazionali, tra cui PostgreSQL, MySQL, SQLite e molti altri. Infine, la struttura modulare di SQLAlchemy consente agli sviluppatori di utilizzare solo le parti della libreria necessarie per il progetto, riducendo così le dipendenze e il carico di lavoro.

Questi pacchetti costituiscono le fondamenta tecnologiche su cui si basa SQML per offrire un'esperienza completa e potente nell'integrazione di machine learning nei database relazionali. Prima di utilizzare SQML, è importante assicurarsi di avere installati tutti questi pacchetti sul proprio sistema.

5.2 Progettazione dei moduli

5.2.1 Dettagli sulla progettazione dei moduli del pacchetto

Il pacchetto adotta un'organizzazione modulare per garantire una gestione chiara e coerente dei vari algoritmi. Ogni tecnica è stata suddivisa in un modulo separato favorendo non solo una migliore organizzazione del codice, ma anche una maggiore coerenza concettuale. In pratica, all'interno di ciascun modulo, è presente una specifica tecnica, rendendo più facile per gli sviluppatori trovare e lavorare con il codice relativo a una specifico metodo di machine learning.

Un aspetto distintivo della progettazione è la dualità nella rappresentazione dei dati. Ogni modulo comprende due classi distinte per ogni algoritmo: una per la rappresentazione relazionale dei dati e una per la rappresentazione COO (Coordinate List). Questo approccio offre agli utenti la flessibilità di scegliere la modalità di rappresentazione dei dati che meglio si adatta alle loro esigenze specifiche di analisi.

Nonostante le differenze nella rappresentazione dei dati, le classi all'interno di ciascun modulo offrono un'interfaccia coerente. Ciò significa che gli utenti possono utilizzare gli stessi metodi e parametri indipendentemente dalla rappresentazione dei dati scelta. Questo rende più intuitivo l'utilizzo del pacchetto e facilita la comprensione del codice.

SQML è progettato per essere estensibile. Ciò significa che è semplice aggiungere nuove tecniche di machine learning o modificare le implementazioni esistenti. Questo permette di mantenere il pacchetto aggiornato con gli ultimi sviluppi nel campo del machine learning, garantendo che gli utenti possano sfruttare le più recenti innovazioni.

Infine, ogni modulo è accompagnato da una documentazione chiara e dettagliata che spiega il funzionamento degli algoritmi, i parametri supportati e fornisce esempi di utilizzo. Questo rende più facile per gli utenti imparare e utilizzare il pacchetto, consentendo loro di sfruttare appieno le sue potenzialità.

5.2.2 Dettagli sulla progettazione delle interfacce dei moduli

Ogni file contiene tre classi correlate, ciascuna con uno scopo specifico all'interno del pacchetto:

- **Classe Builder:** questa classe è responsabile della creazione dinamica di istanze di una delle due altre classi, in base alla densità dei dati su cui fare la previsione.
- **Classe per la rappresentazione relazionale:** istanziata dalla classe Builder nel caso i dati su cui effettuare la previsione siano densi.
- **Classe per la rappresentazione COO:** istanziata dalla classe Builder nel caso i dati su cui effettuare la previsione siano sparsi.

Le due classi che implementano il metodo di machine learning si basano su un'interfaccia comune che mira a garantire una coerenza e facilità d'uso. Questa interfaccia, si articola in tre principali componenti:

Innanzitutto, entrambe le classi dispongono di un costruttore. Questo elemento è responsabile di inizializzare l'oggetto della classe e, se necessario, può essere utilizzato per inizializzare i parametri del modello. È il punto di partenza cruciale per creare e configurare correttamente l'istanza del modello.

Entrambe le classi implementano il metodo *fit*. Questo metodo è utilizzato per l'addestramento del modello. Durante tale processo, i parametri del modello vengono ottimizzati in base ai dati di addestramento e questi parametri vengono salvati in una tabella dedicata all'interno del database, pronti per l'utilizzo futuro.

Infine, dispongono di un metodo *predict*. Questo metodo è di fondamentale importanza per effettuare previsioni sui dati di test. Utilizzando i parametri del modello addestrato precedentemente salvati nella tabella del database, il metodo genera previsioni precise e affidabili, utilizzando i dati di test presenti nella tabella opportuna. Questo processo assicura una semplicità d'uso per gli utenti del pacchetto, consentendo loro di ottenere previsioni accurate senza complicazioni eccessive.

In sintesi, questa interfaccia comune fornisce una base solida e intuitiva per l'utilizzo delle classi di machine learning nel pacchetto. Grazie alla sua struttura chiara e ben definita, gli utenti possono facilmente addestrare e utilizzare modelli di machine learning, sfruttando al massimo le potenzialità del pacchetto senza incontrare ostacoli o difficoltà nel processo.

5.3 Sviluppo

In questa sezione, ci concentriamo sul codice, illustrando come il pacchetto fornisca metodi di machine learning integrati. Utilizziamo l'implementazione del modulo di regressione logistica come esempio esplicativo.

5.3.1 Classe Builder

```
class LogisticRegressionBuilder:
    @staticmethod
    def build(dense: bool):
        return LogisticRegressionRE() if dense else LogisticRegressionCOO()
```

Utilizzando un metodo statico `build`, la classe `Builder` determina se i dati sono densi o sparsi e istanzia la classe corretta, consentendo un'implementazione flessibile e ottimizzata per le caratteristiche specifiche dei dati.

Nel codice fornito, la classe *LogisticRegressionBuilder* implementa un metodo statico chiamato *build*. Questo metodo accetta un argomento booleano *dense*, che indica se i dati sono densi (True) o sparsi (False). A seconda del valore di *dense*, il metodo istanzia e restituisce un'istanza della classe *LogisticRegressionRE* se i dati sono densi, altrimenti restituisce un'istanza della classe *LogisticRegressionCOO* se i dati sono sparsi.

Questo approccio consente di gestire in modo ottimale dati con diverse caratteristiche di densità. Ad esempio, se i dati sono densi, si preferisce utilizzare *LogisticRegressionRE*, ottimizzata per questo tipo di dati; se i dati sono sparsi, si utilizza invece *LogisticRegressionCOO*, ottimizzata per questa specifica situazione. In questo modo, il codice fornisce una soluzione flessibile per la costruzione del modello di regressione logistica, consentendo di adattarsi efficacemente alle caratteristiche dei dati in ingresso.

5.3.2 Definizione e costruttori delle Classi `LogisticRegressionRE` e `LogisticRegressionCOO`

```
class LogisticRegressionRE(LogisticRegression):
```

```
    def __init__(self):
        super().__init__()
```

```
class LogisticRegressionCOO(LogisticRegression):
```

```
    def __init__(self):
        super().__init__()
```

Le classi *LogisticRegressionRE* e *LogisticRegressionCOO* sono entrambe sottoclassi di *LogisticRegression* di scikit-learn, il che significa che avranno accesso a tutti i metodi e attributi definiti nella classe padre. Questo approccio è fondamentale perché consente alle nuove classi di sfruttare l'implementazione esistente della regressione logistica fornita da scikit-learn, estendendola o adattandola per poi integrarla in SQL. Effettivamente, il costruttore delle sottoclassi invoca il costruttore della classe padre assicurando che l'inizializzazione di quest'ultima venga eseguita correttamente, garantendo che gli attributi e i metodi della classe base siano disponibili anche nelle sottoclassi.

5.3.2 Addestramento

Il metodo *fit* è responsabile della fase di addestramento del modello.

Tale metodo rimane molto simile in entrambe le classi, con la differenza di come verrà rappresentato il modello addestrato nel database.

Definizione

```
def fit(self,X,Y,engine, names):
```

Il metodo contiene i seguenti parametri:

- *self*: Questo parametro si riferisce all'istanza della classe stessa e consente al metodo di accedere agli attributi e ai metodi dell'oggetto.
- *X*: Questo è l'array bidimensionale dei dati di input, dove ogni riga rappresenta un'istanza e ogni colonna rappresenta una feature.
- *Y*: Questo è l'array unidimensionale dei target corrispondenti ai dati di input X. Deve avere la stessa lunghezza di X.
- *engine*: Questo parametro rappresenta l'engine SQLAlchemy che viene utilizzato per la connessione al database. Viene passato al fine di poter interagire con il database per memorizzare i risultati dell'addestramento.
- *names*: Questo è un elenco dei nomi delle colonne da utilizzare per etichettare le variabili nel database. Questo è utile quando vogliamo identificare quali colonne corrispondono a quali feature nei dati di input.

Addestramento del modello

```
clf = super().fit(X,Y)
```

Il modello di regressione logistica viene addestrato utilizzando i dati di input X e i relativi target Y. Questo passaggio è gestito invocando il metodo `fit` della classe padre *LogisticRegression* di `scikit-learn`.

Estrazione dei parametri

Dopo aver addestrato il modello, vengono estratti i parametri necessari per fare previsione sui nuovi dati, in questo caso i coefficienti e il bias.

Nel caso di regressione logistica multiclasse, queste due istruzioni sono sufficienti.

```
coeffs = clf.coef_  
bias = clf.intercept_
```

Nel caso invece fosse binaria, è necessaria un'ulteriore manipolazione. Bisogna quindi distinguere il caso.

```
if coeffs.shape[0] == 1:  
    coeffs = [coeffs[0], [-value for value in coeffs[0]]]  
    bias = np.array([bias[0], -bias[0]])
```

Quando il modello di regressione logistica ha solo una classe o un'etichetta di output, il modello restituirà i coefficienti solo per la classe positiva, trascurando la classe negativa.

Per garantire che il nostro modello possa gestire sia la classe positiva che quella negativa, bisogna creare dei coefficienti e un termine bias per la classe negativa. Di conseguenza, si duplicano i coefficienti ottenuti e si negano, uguale per il termine bias. Questo assicura che il modello sia in grado di produrre probabilità valide per entrambe le classi, sia positiva che negativa.

Rappresentazione del modello addestrato nel database

A questo punto, bisogna rappresentare il modello nel database attraverso la creazione di un'opportuna tabella e l'inserimento dei dati, sfruttando SQLAlchemy. Questa parte si può dividere in tre fasi principali. Prima di tutto, viene creata la struttura della tabella che conterrà i

dati relativi ai coefficienti e al bias del modello. Questa fase assicura che il database sia pronto ad accettare i dati del modello.

Successivamente, c'è l'inserimento dei dati nella tabella del database. In questa fase. Questo passaggio è cruciale per memorizzare i parametri del modello nel database in modo da poterli utilizzare successivamente durante la fase di predizione. Infine, c'è il salvataggio delle modifiche. Questo viene fatto eseguendo il commit della transazione. In questo modo, le modifiche apportate alla tabella del database diventano permanenti. Questo passaggio assicura che i dati del modello siano memorizzati in modo sicuro e accessibili per le operazioni di predizione future.

Nelle due classi, si verifica una differenza nella rappresentazione del modello nel database, al fine di adattare il modello per la fase successiva di predizione.

Classe LogisticRegressionRE:

```

        metadata = MetaData()
training = Table(
    "training",
    metadata,
    Column('id', Integer, primary_key=True),
    *[Column(f'{i}', Float) for i in names],
    Column('bias', Float)
)
metadata.create_all(engine)
session = Session(engine)
# INSERIMENTO DATI
for x_values, y_value in zip(coeffs, bias):
    row_data = dict(zip([f'{i}' for i in names] + ['bias'],
                        list(x_values) + [float(y_value)]))
    session.execute(training.insert().values(row_data))
session.commit()

```


Classe LogisticRegressionCOO:

```

metadata = MetaData()
training = Table(
    "training",
    metadata,
    Column('row', Integer, primary_key=True),
    Column('column', Integer, primary_key=True),
    Column('value', Float, primary_key=True),
    Column('bias', Float)
)
metadata.create_all(engine)
session = Session(engine)
righe, colonne = coeffs.shape
    # INSERIMENTO DATI
for i in range(righe):
    for j in range(colonne):
        insert_query = text(
            """
            INSERT INTO training ("row", "column", "value",
                                "bias")
            VALUES (:row, :column, :value, :bias)
            """
        )
        session.execute(insert_query, {"row": i, "column": j,
                                       "value": coeffs[i][j],
                                       "bias": int(bias[i])})
session.commit()

```

In sintesi, questa parte del codice gestisce la creazione della struttura della tabella del database, l'inserimento dei dati relativi ai coefficienti e al bias del modello nella tabella e il salvataggio

delle modifiche apportate al database. Questo processo è essenziale per mantenere e utilizzare i parametri del modello in modo efficace durante la fase di predizione.

5.3.3 Predizione

La fase di predizione è il processo in cui utilizziamo il modello addestrato per fare previsioni su nuovi dati. In questo contesto, la fase di predizione avviene principalmente attraverso il metodo *predict()*, che non fa altro che eseguire una query SQL sulle tabelle del database che contengono i parametri del modello addestrato e i dati su cui effettuare la previsione.

Poiché le due classi rappresentano due formati di dati diversi (relazionale e COO), le query di predizione dovranno essere adattate di conseguenza. Nonostante la logica alla base della predizione rimanga la stessa, la struttura e la sintassi delle query SQL saranno differenti per ciascuna classe al fine di elaborare correttamente i dati e produrre le previsioni desiderate.

Classe **LogisticRegressionRE**:

```
def predict(self, table, engine, names):

    connection = engine.connect()

    microquery = "(l.bias + "

    for word in names:
        microquery += f"(l.{word} * {table}.{word})+"
    microquery = microquery[:-1]+"")

    query = f"""

    WITH step AS (
```

```

SELECT l.id as label, {table}.id as id,
       1/(1+POW({math.e},-{microquery})) as prob
FROM training as l, {table}
) SELECT label-1, id FROM step WHERE prob >=
      (SELECT MAX(prob)
       FROM step as s WHERE s.id = step.id) ORDER BY id
'''
query = text(query)
results = connection.execute(query)

```

Nel processo di previsione, la prima tappa è stabilire una connessione al motore del database, utilizzando il metodo *connect()*. Questo passaggio è fondamentale per interagire con il database e recuperare i dati necessari per effettuare le previsioni.

Successivamente, viene costruita la parte della query relativa al calcolo delle probabilità che avviene iterando attraverso la lista *names*, che contiene i nomi delle colonne rappresentanti le feature del modello. Questo approccio è adottato perché non si conosce a priori la dimensione o la struttura della tabella su cui verrà effettuata la predizione. Utilizzando un ciclo che opera attraverso i nomi delle feature, possiamo adattare la query a qualsiasi tabella di dati fornita come input (dal parametro *table*).

Successivamente, viene composta la query SQL completa che sarà in grado di calcolare la probabilità di appartenenza a ciascuna classe per ogni osservazione nel dataset di test.

Infine, la query viene eseguita attraverso il metodo *execute()* dell'oggetto di connessione, il che consente di inviare la query al database e ottenere i risultati. I risultati restituiti dalla query rappresentano le previsioni effettuate dal modello di regressione logistica sui dati di test.

Classe LogisticRegressionCOO:

```
query = f"""
    WITH step AS (
        SELECT l.row as label, {table}.row as id, 1/(1+POW( {math.e}, -(l.bias+SUM(l.value* {table}.value))))
    as prob
        FROM training as l, {table}
        WHERE l.column = {table}.column
        GROUP BY l.row, {table}.row, l.bias
    ) SELECT label, id FROM step WHERE prob >= (SELECT MAX(prob) from step as s WHERE s.id =
step.id) ORDER BY id
    """
```

In questo caso, la logica di predizione rimane la stessa, ma la query viene costruita in modo da adattarsi alla struttura dei dati rappresentati da questa forma di memorizzazione. La differenza principale risiede nel modo in cui i coefficienti del modello sono organizzati e utilizzati per effettuare le previsioni, ovvero come una lista di triplette (row, column, value), dove ogni tripletta rappresenta un coefficiente non nullo del modello. Quindi, la query per la predizione deve iterare su queste triplette e utilizzare i valori corrispondenti delle features per calcolare le probabilità di appartenenza alle classi.

Capitolo 6

Valutazione

La valutazione dell'efficacia dei metodi di machine learning integrati nel pacchetto riveste un ruolo cruciale nel determinare la loro utilità e affidabilità nell'ambito dell'analisi dei dati direttamente in SQL.

Durante la valutazione, sono stati considerati due aspetti chiave. In primo luogo, è stata valutata l'accuratezza predittiva dei modelli, al fine di determinare quanto bene i modelli riescano a fare previsioni accurate su dati non visti. In secondo luogo, sono state analizzate le prestazioni dei modelli in termini di tempo di predizione. Questo consente di valutare l'efficienza dei metodi integrati e di identificare eventuali punti critici che richiedono ottimizzazione.

Complessivamente, la valutazione permette di ottenere una panoramica completa delle capacità e delle prestazioni delle tecniche di machine learning. Questo fornisce informazioni preziose per migliorare e ottimizzare ulteriormente il pacchetto, garantendo che possa soddisfare al meglio le esigenze degli utenti e fornire risultati affidabili e accurati nell'ambito dell'analisi dei dati direttamente in SQL.

6.1 Dataset utilizzati per la valutazione

Per valutare l'efficacia dei metodi di machine learning integrati nel pacchetto, sono stati utilizzati due dataset rappresentativi: l'Iris dataset, e lo Zoo dataset.

I dataset sono stati scelti opportunamente per essere rappresentativi dei due tipi di formati supportati dal pacchetto (COO e relazionale). I test sono stati effettuati con entrambi i dataset in entrambi i formati disponibili. Complessivamente, l'uso di due dataset rappresentativi con

diverse caratteristiche di densità dei dati e la valutazione delle prestazioni in entrambi i formati ha permesso di ottenere una valutazione completa dell'efficacia dei metodi integrati nel pacchetto.

6.1.1 Iris dataset

L'Iris dataset è un dataset ampiamente utilizzato in ambito di classificazione. Contiene informazioni su tre specie di fiori Iris, con misurazioni relative a lunghezza e larghezza dei sepali e dei petali, per un totale di 4 features. Questo dataset è composto da dati densi, in quanto non presenta valori nulli o mancanti, di conseguenza si presta bene ad un formato dati relazionale.

Nel formato relazionale ogni riga corrisponde a un fiore e ogni colonna rappresenta una caratteristica misurata (feature).

id	sepal_length	sepal_width	petal_length	petal_width
1	5.1	3.5	1.4	0.2
2	4.9	3	1.4	0.2
3	4.7	3.2	1.3	0.2
4	4.6	3.1	1.5	0.2
5	5	3.6	1.4	0.2

Nel formato COO ogni riga rappresenta una feature, identificata dalla tripla $\langle \text{row}, \text{column}, \text{value} \rangle$. L'insieme delle triple che condividono lo stesso indice di riga, rappresentano un fiore.

row integer	column integer	value numeric
0	0	5.1
0	1	3.5
0	2	1.4
0	3	0.2
1	0	4.9

6.1.2 Zoo dataset

Lo zoo dataset descrive varie specie di animali e include attributi come il numero di zampe, il tipo di riproduzione, per un totale di 16 features. Questo dataset è composto da dati sparsi, in quanto non presenta valori nulli o mancanti, di conseguenza si presta bene ad un formato dati COO.

Nel formato relazionale il dataset evidenzia la propria natura sparsa, con molte voci pari a zero.

1	1	0	0	1	0
2	1	0	0	1	0
3	0	0	1	0	0
4	1	0	0	1	0
5	1	0	0	1	0
6	1	0	0	1	0
7	1	0	0	1	0
8	0	0	1	0	0
9	0	0	1	0	0

Nel formato COO, vengono visualizzati solo i dati che non sono nulli.

 row integer	 column integer	 value numeric
0	0	1
0	3	1
0	6	1
0	7	1
0	8	1
0	9	1

6.2 Valutazione dell'accuratezza predittiva

La valutazione dell'accuratezza predittiva dei modelli rappresenta un aspetto cruciale nello sviluppo del pacchetto. Questo processo consente di determinare se i classificatori stanno effettuando previsioni corrette e affidabili, fornendo una valutazione fondamentale della loro efficacia complessiva.

6.2.1 Metodo di valutazione

Per valutare l'accuratezza dei modelli, in primo luogo vengono effettuate previsioni utilizzando i modelli stessi. Queste previsioni vengono poi confrontate con le etichette corrette associate ai dati di test. Il numero di previsioni corrette viene quindi diviso per il numero totale di previsioni effettuate, fornendo così un indice di accuratezza. Questo processo ci fornisce un'indicazione chiara delle precisioni dei classificatori, consentendo di comprendere se le previsioni sono accurate e affidabili.

6.2.2 Confronto con scikit-learn

Successivamente, è importante confrontare le percentuali di accuratezza dei classificatori SQML con quelle di scikit-learn, una libreria ampiamente riconosciuta nel campo del machine learning. Il confronto delle performance permette di valutare il livello di affidabilità e precisione dei modelli SQML così da ottenere una visione chiara della qualità dei modelli SQML rispetto a uno standard di riferimento nel settore.

6.2.3 Risultati

I risultati ottenuti dimostrano che le precisioni per ogni tipo di rappresentazione e per ogni tipo di tecnica integrata sono elevate e uguali tra loro. Questo suggerisce che le tecniche implementate

sono efficaci e producono risultati affidabili, in linea con un framework ampiamente utilizzato come scikit-learn, di conseguenza la logica predittiva è stata correttamente implementata in SQL.

6.3 Valutazione delle prestazioni

La valutazione delle prestazioni non si limita solo alla precisione predittiva, ma comprende anche la valutazione delle prestazioni in termini di tempo. Misurare il tempo di predizione è cruciale per comprendere quanto velocemente un modello può fare previsioni su nuovi dati.

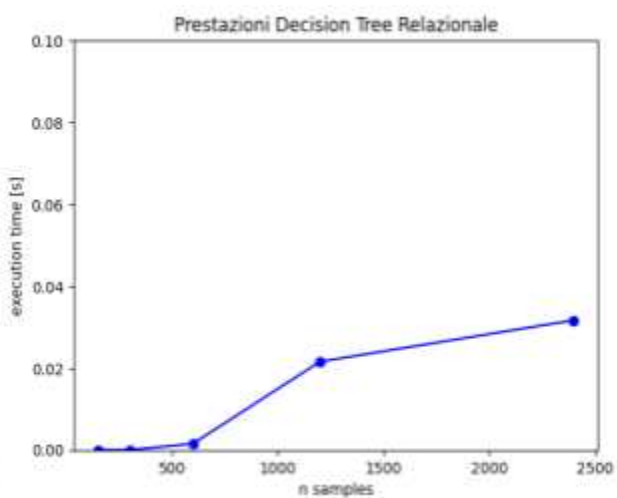
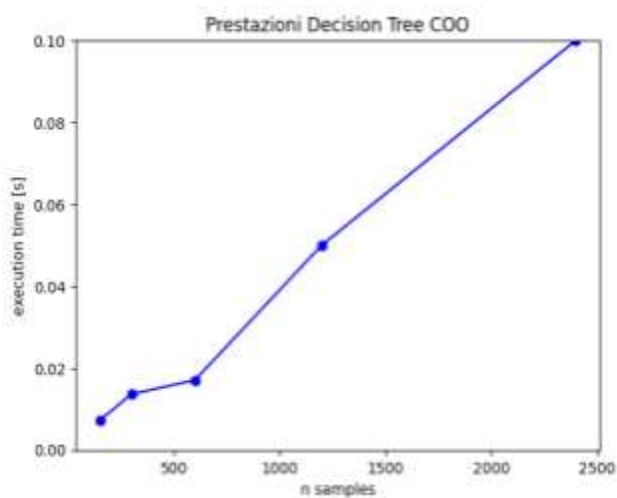
6.3.1 Metodo di valutazione

Il tempo di predizione è stato registrato per ogni modello, sia nel formato SQML.RE che SQML.COO, utilizzando dataset di dimensioni crescenti. In particolare, è stato misurato il tempo dell'operazione di predizione, cioè l'esecuzione della query contenente la logica predittiva. Questo ha consentito di valutare l'aumento del tempo di predizione al crescere delle dimensioni del dataset, offrendo così informazioni sulla scalabilità dei modelli integrati. Entrambi i formati, sono stati testati su dataset densi e sparsi al fine di comprendere come le diverse rappresentazioni dei dati influenzino il tempo di predizione in relazione al tipo di dataset.

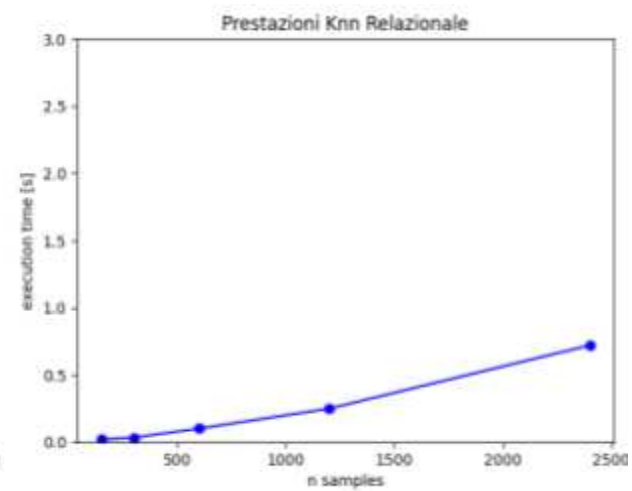
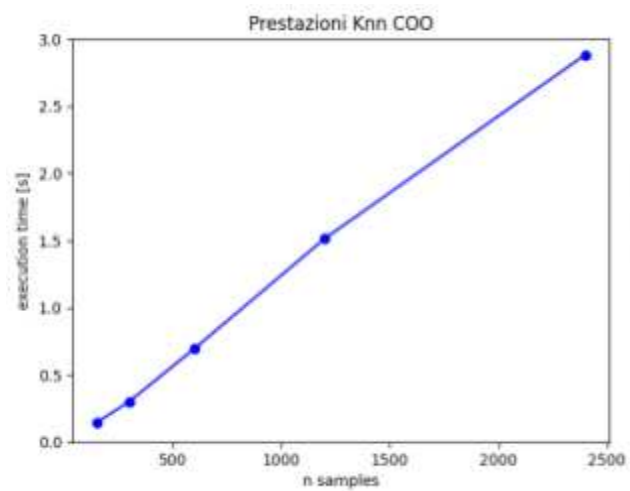
6.3.2 Grafici dei tempi

Al fine di analizzare le prestazioni dei modelli, sono stati realizzati dei grafici che mostrano il tempo di predizione in relazione alle dimensioni del dataset e al tipo di rappresentazione dei dati. Partendo dall'analisi delle prestazioni su dati densi, quindi sul primo dataset precedentemente descritto, seguono grafici esplicativi delle prestazioni di ogni metodo integrato.

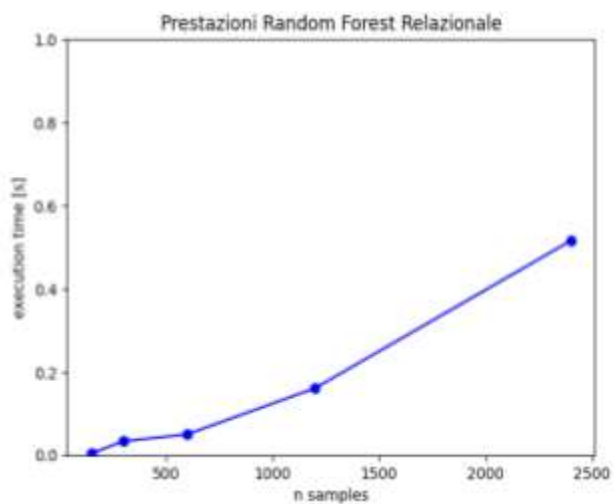
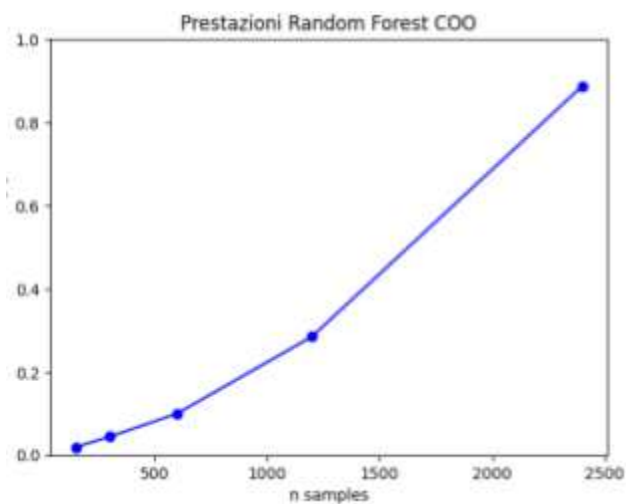
Decision Tree



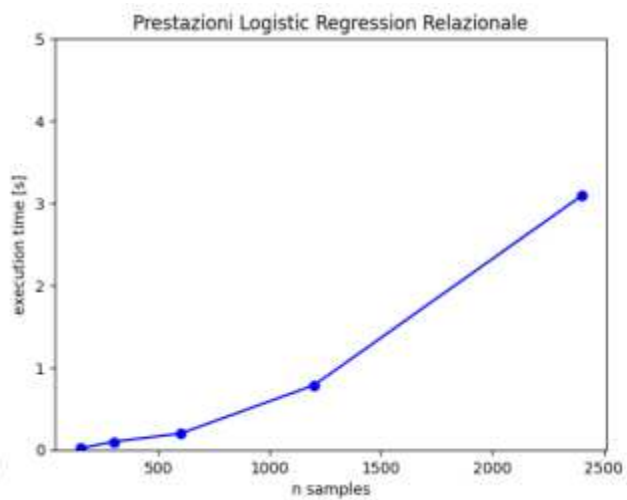
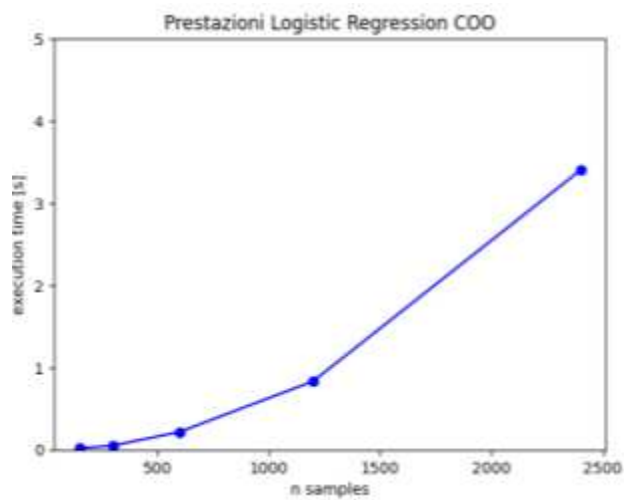
Knn



Random Forest



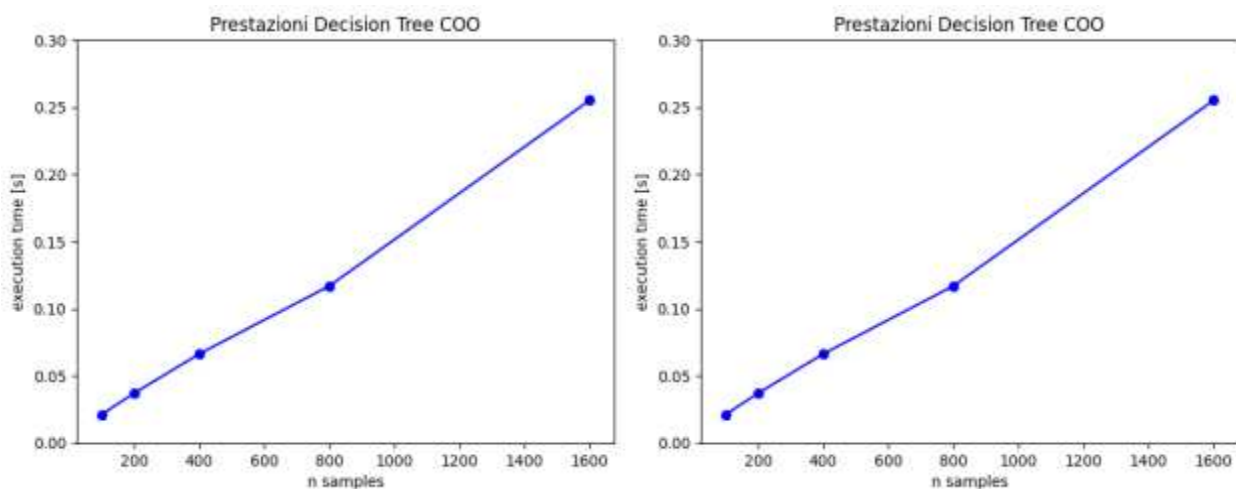
Logistic Regression



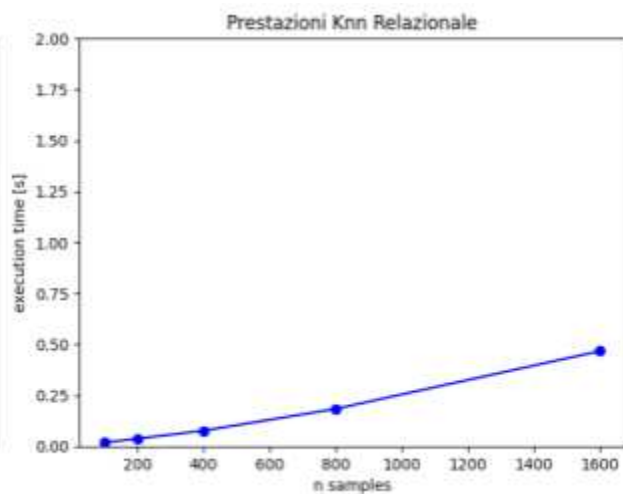
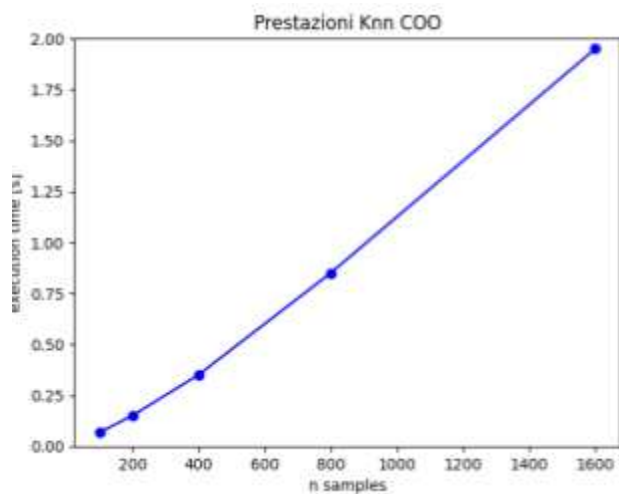
Come evidenziato dai grafici, il formato relazionale si dimostra più efficace sui dati densi, con prestazioni superiori. È interessante notare che tutti gli algoritmi mostrano prestazioni efficienti, in particolare gli alberi decisionali che si distinguono per le loro prestazioni eccellenti, di pochi centesimi di secondo.

Le valutazioni delle prestazioni sono state effettuate anche sul secondo dataset presentato, un caso di dataset sparso.

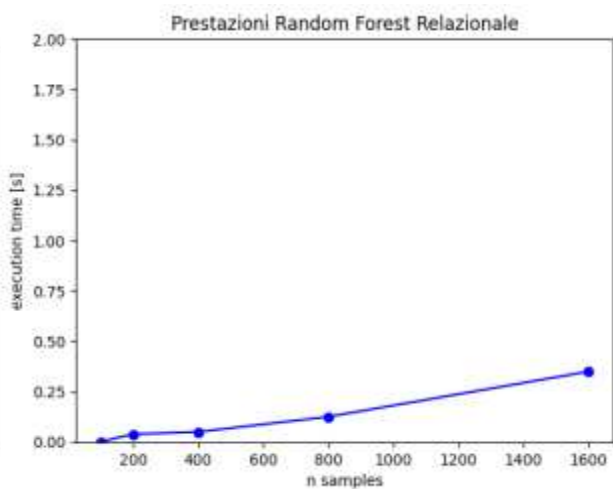
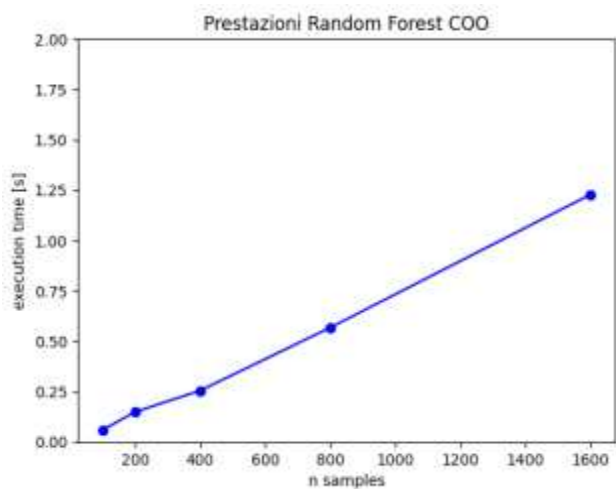
Decision Tree



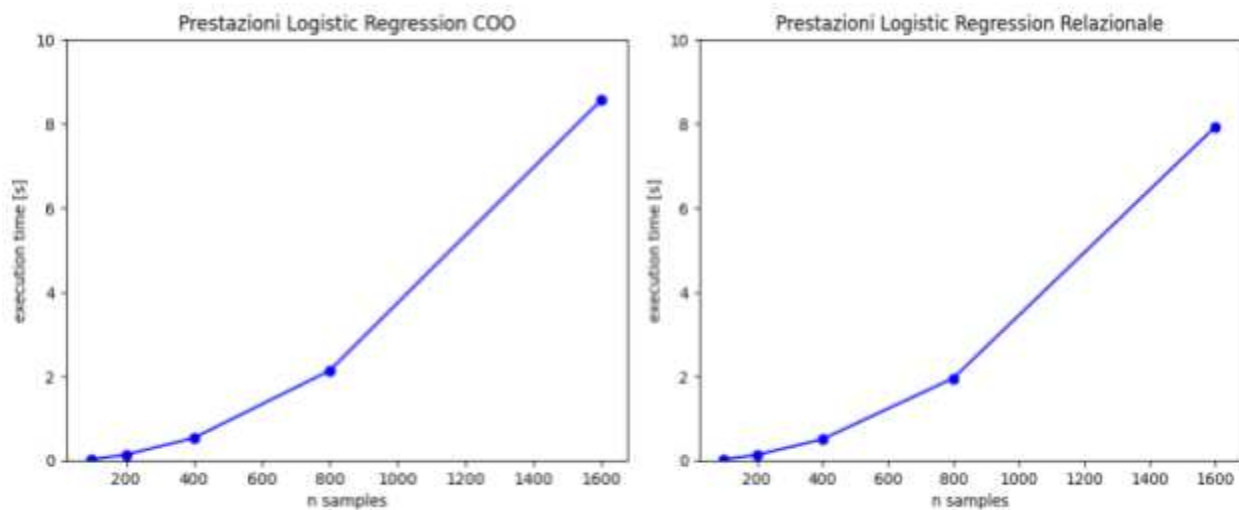
Knn



Random Forest



Logistic Regression



Le prestazioni dei formati COO risultano essere più vicine al formato relazionale rispetto al caso dei dati densi, nonostante ci si aspettasse che fossero migliori al formato relazionale. Questo potrebbe essere dovuto al fatto che il dataset non è abbastanza sparso, alla rappresentazione del modello addestrato nella tabella o alla non ottimale ottimizzazione della query di predizione.

Capitolo 7

Conclusione e sviluppi futuri

Il lavoro svolto durante lo stage ha portato alla realizzazione di un pacchetto Python denominato SQML, concepito per l'integrazione dei metodi di machine learning in SQL. L'obiettivo principale di SQML era quello di sfruttare appieno i vantaggi offerti dai database relazionali, trasferendo la logica predittiva degli algoritmi di machine learning direttamente all'interno di tali ambienti, direttamente sui dati archiviati, eliminando così la necessità di eseguire operazioni d'estrazione.

Inoltre, attraverso un'attenta valutazione dell'accuratezza dei modelli integrati, si è confermato che essi mantengono livelli di precisione simili alle implementazioni equivalenti presenti nella libreria scikit-learn. Questo ha confermato l'efficacia dei metodi integrati nel pacchetto e ha fornito una solida base per il loro utilizzo in contesti reali.

Tra gli svantaggi di questo pacchetto, va considerato il fatto che non tutte le performance degli algoritmi integrati potrebbero raggiungere gli standard delle tradizionali librerie di machine learning come scikit-learn. Tuttavia, è importante sottolineare che queste performance, sono comunque da considerarsi accettabili e dipendono dal DBMS in uso.

Il pacchetto si adatta al meglio in base al tipo dei dati, sparsi o densi. Questo adattamento consente all'utente di non preoccuparsi del formato con il quale i dati sono stati salvati, potendo essere gestito in autonomia dal pacchetto.

I vantaggi derivanti dall'utilizzo di SQML sono molteplici e significativi. In particolare, il pacchetto offre una significativa riduzione dei costi associati all'estrazione e alla manipolazione dei dati dal database. Grazie alla possibilità di eseguire il calcolo direttamente all'interno del database, SQML elimina la necessità di trasferire grandi quantità di dati o di eseguire complesse operazioni di pre-elaborazione, migliorando l'efficienza complessiva del processo.

Inoltre, SQML garantisce una maggiore privacy dei dati trattati. In contesti in cui i dati sono di natura sensibile o soggetti a rigorose normative sulla privacy, l'esecuzione dei modelli di machine learning direttamente nel database evita l'esposizione dei dati sensibili al di fuori dell'ambiente controllato.

Tuttavia, vi sono ancora ampie opportunità per lo sviluppo futuro del pacchetto. Una direzione promettente potrebbe essere l'estensione della gamma di algoritmi di machine learning integrati. Introdurre nuovi algoritmi potrebbe consentire agli utenti di affrontare una maggiore varietà di problemi e di ottenere risultati più accurati ed efficienti. Oltre all'espansione delle capacità di machine learning, c'è spazio per ottimizzare ulteriormente il pacchetto per gestire grandi volumi di dati in modo più efficiente, in particolare si potrebbe ottimizzare le prestazioni dei classificatori adibiti al formato COO per sfruttare a pieno la rappresentazione.

In conclusione, sebbene le prestazioni di SQML possano risultare potenzialmente inferiori rispetto agli approcci tradizionali, è essenziale riconoscere i suoi vantaggi significativi. SQML offre una riduzione dei costi, un'efficienza operativa migliorata e un livello superiore di sicurezza dei dati. Inoltre, è importante considerare le molteplici possibilità di sviluppo futuro che potrebbero contribuire a migliorare e correggere eventuali difetti nel pacchetto.

BIBLIOGRAFIA

- [1] Cumin, J., Petit, J. M., Scuturici, V. M., & Surdu, S. (2017, March). Data exploration with sql using machine learning techniques. In *International Conference on Extending Database Technology-EDBT* (pp. 96-107).
- [2] Kaufmann, M., Stechschulte, G., & Huber, A. (2021). Efficient and Accurate In-Database Machine Learning with SQL Code Generation in Python. arXiv preprint arXiv:2104.03224.
- [3] Jankov, Dimitrije, et al. "Declarative recursive computation on an rdbms, or, why you should use a database for distributed machine learning." *arXiv preprint arXiv:1904.11121* (2019).
- [4] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. Tensorflow: A system for large-scale machine learning. In OSDI, pages 265–283, 2016.
- [5] Makrynioti, N., Ley-Wild, R., & Vassalos, V. (2021, June). Machine learning in SQL by translation to TensorFlow. In *Proceedings of the Fifth Workshop on Data Management for End-To-End Machine Learning* (pp. 1-11).
- [6] Joseph M. Hellerstein, Christopher Ré, Florian Schoppmann, Daisy Zhe Wang, Eugene Fratkin, Aleksander Gorajek, Kee Siong Ng, Caleb Welton, Xixuan Feng, Kun Li, and Arun Kumar. 2012. The MADlib Analytics Library: Or MAD Skills, the SQL. Proc. VLDB Endow. 5, 12 (Aug. 2012), 1700–1711. <https://doi.org/10.14778/2367502.2367510>
- [7] Maximilian Schleich, Dan Olteanu, and Radu Ciucanu. 2016. Learning Linear Regression Models over Factorized Joins. In Proceedings of the 2016 International Conference on Management of Data (San Francisco, California, USA) (SIGMOD '16). ACM, New York, NY, USA, 3–18.
- [8] Ying Zhang, Martin Kersten, and Stefan Manegold. 2013. SciQL: Array Data Processing Inside an RDBMS. In Proceedings of the 2013 ACM SIGMOD International Conference on Management of Data (New York, New York, USA) (SIGMOD '13). ACM, New York, NY, USA, 1049–1052.

- [9] Sarawagi, S., Thomas, S., & Agrawal, R. (1998). Integrating association rule mining with relational database systems: Alternatives and implications. *ACM SIGMOD Record*, 27(2), 343-354.
- [10] Del Buono, F., Paganelli, M., Sottovia, P., Interlandi, M., & Guerra, F. (2021, June). Transforming ML predictive pipelines into SQL with MASQ. In *Proceedings of the 2021 International Conference on Management of Data* (pp. 2696-2700).
- [11] Luo, S., Gao, Z. J., Gubanov, M., Perez, L. L., & Jermaine, C. (2018). Scalable linear algebra on a relational database system. *ACM SIGMOD Record*, 47(1), 24-31.
- [12] S. Chaudhuri. An overview of query optimization in relational systems. In PODS, pages 34–43, 1998.
- [13] Vigliano, L. "Sistemi di Basi di Dati-Fondamenti e Complementi 7/Ed." (2018).
- [14] Giesser, P., Stechschulte, G., da Costa Vaz, A., & Kaufmann, M. (2021, December). Implementing efficient and scalable in-database linear regression in SQL. In *2021 IEEE International Conference on Big Data (Big Data)* (pp. 5125-5132). IEEE.
- [15] Blacher, M., Giesen, J., Laue, S., Klaus, J., & Leis, V. (2022). Machine learning, linear algebra, and more: Is SQL all you need?
- [16] D. Fetter, "Lists and recursion and trees, oh my!." FOSDEM: Free and Open Source Software Developers' European Meeting, 2010.
- [17] Schule, M., Lang, H., Springer, M., Kemper, A., Neumann, T., & Gunnemann, S. (2021, July). In-database machine learning with SQL on GPUs. In *Proceedings of the 33rd International Conference on Scientific and Statistical Database Management* (pp. 25-36).
- [18] Brucato, M., Beltran, J. F., Abouzied, A., & Meliou, A. (2015). Scalable package queries in relational database systems. *arXiv preprint arXiv:1512.03564*.
- [19] Großmann, C., & Schildgen, J. (1989). Integrating Machine Learning into SQL with Exasol. *Software Engineer*, 56951-48.
- [20] D'silva, J. V., De Moor, F., & Kemme, B. (2018). AIDA: Abstraction for advanced in-database analytics. *Proceedings of the VLDB Endowment*, 11(11), 1400-1413.
- [21] Wang, Y., Yang, Y., Zhu, W., Wu, Y., Yan, X., Liu, Y., ... & Tang, Y. (2020). SQLflow: a bridge between SQL and machine learning. *arXiv preprint arXiv:2001.06846*.

- [22] Pooch, U. W., & Nieder, A. (1973). A survey of indexing techniques for sparse matrices. *ACM Computing Surveys (CSUR)*, 5(2), 109-133.
- [23] Numpy. URL: <https://numpy.org/doc/stable/>
- [24] Pandas. URL: <https://pandas.pydata.org/docs/>
- [25] SQLAlchemy. URL: <https://www.sqlalchemy.org/>